

Document Proof Storage FAQ

This document summarizes the storage mechanism and limitations of student document proofs in the project.

Question 1: Where are we storing the document proofs entered by students? (PostgreSQL cannot store documents/images/pdfs)

Answer:

The application uses a **hybrid storage pattern** where the actual files are stored on the server's filesystem, and only the reference URLs are stored in the PostgreSQL database.

1. File Storage (Server Filesystem):

- Upload requests are handled by the API route at [`src/app/api/upload/route.ts`] (file:///c:/Users/Sachin%20Jha/Desktop/Everything/SIH2026/src/app/api/upload/route.ts).
- The file is saved directly to the local directory on the server: `public/uploads/` (resolved relative to the project root as `join(process.cwd(), 'public', 'uploads')`).
- A unique filename is created by prefixing the cleaned file name with `Date.now()` to prevent overwriting.

2. Database Storage (PostgreSQL via Prisma):

- Instead of storing binary data (BLOBs), PostgreSQL stores the public relative URL string (e.g., `"/uploads/1723984859000-my_certificate.pdf"`).
- In [`prisma/schema.prisma`] (file:///c:/Users/Sachin%20Jha/Desktop/Everything/SIH2026/prisma/schema.prisma), these are reference fields:
 - `Activity.evidenceUrl`: Stores the link to the uploaded proof file.
 - `ProjectContribution.projectEvidence`: Stores the link to the project documentation/evidence file.
 - `Evidence.storagePath`: Stores the relative path to the uploaded file.

Question 2: What is the limit of the server's local filesystem?

Answer:

The storage and upload limits are constrained by four distinct boundaries:

1. Client-Side Validation (5MB Limit):

- The user interface in [`PassportClient.tsx`] (file:///c:/Users/Sachin%20Jha/Desktop/Everything/SIH2026/src/app/student/passport/PassportClient.tsx) enforces a maximum file size check of **5MB** before upload:

```
alert('Evidence file size must not exceed 5MB.');
```

2. Server/Hosting Platform Request Limit (4.5MB - 5MB Limit):

- If deployed on **Vercel**, serverless functions have a hard body payload size limit of **4.5MB**. Any upload request exceeding this size will return a **413 Payload Too Large** error.
- For standalone Node.js servers, Next.js defaults to a body size limit of **4MB** unless configured otherwise.

3. Physical Server Disk Capacity (Hardware Limit):

- The local directory **public/uploads** uses the storage of the hosting virtual machine/hard drive. If the disk fills up, the server will crash or fail to write new uploads.

4. Ephemeral Filesystem Reset (Infrastructure Limitation):

- If hosted on serverless or cloud container infrastructure (e.g., Vercel, Netlify, Heroku, or basic AWS ECS without persistent EFS mounts), the filesystem is **ephemeral**.
- Whenever the server container restarts or redeploys, all files stored in **public/uploads** are **wiped out/deleted**.
- **Recommendation:** For production environments, configure the application to upload files to an external persistent object storage service such as **AWS S3**, **Supabase Storage**, or **Vercel Blob** instead of the local server disk.